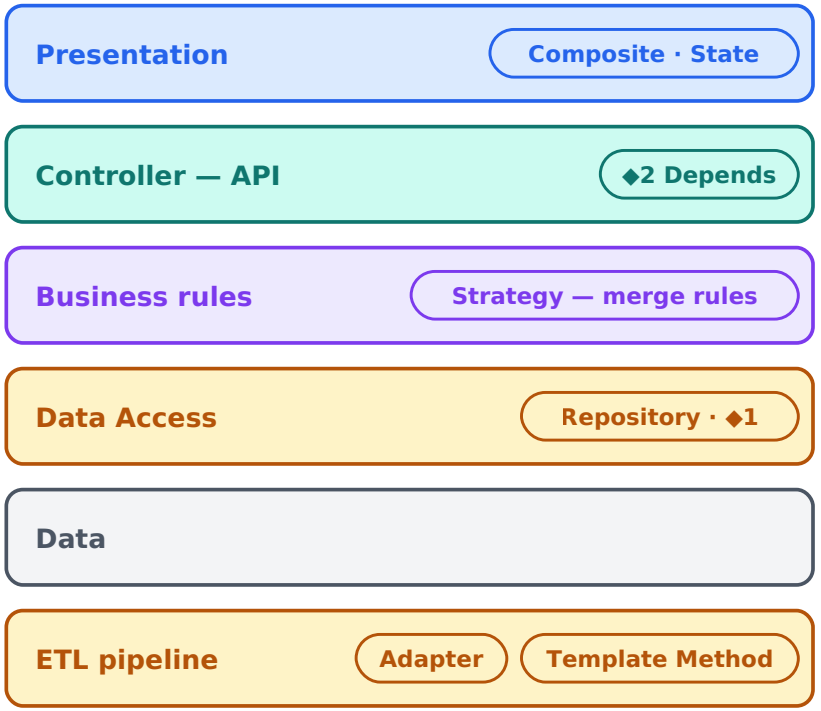
Patterns and Modular Code

Reuse the pattern the code already shows. Extend by subclassing a parent. Compose in a job or an entry file. Depend on the parent, never on a child.

A **parent** is an abstract base class. It fixes the shape of a part; a child fills in one case. Callers use the parent type, so a child can swap without caller changes. That swap point is an inversion of control seam — the ◆ markers in the architecture views.

Pattern names below come from their sources, not from this guide: **GoF** is Gamma, Helm, Johnson, and Vlissides, “Design Patterns” (1994). **Repository** is Fowler, “Patterns of Enterprise Application Architecture” (2002). **Dependency Injection** is Fowler (2004). Rows marked **React idiom** have no GoF name and get none.

Pattern	What it solves	Where it lives	Stub
Adapter · GoF	Converts the interface of a class into another interface clients expect	A normaliser per source converts source fields to the contract shape	1
Template Method · GoF	Defines the skeleton of an algorithm; subclasses provide the steps	Jobs: fetch, normalize, write · MergeProcessor: read, match, merge, write. The parents declare the steps	1
Strategy · GoF	Defines a family of algorithms, encapsulates each one, makes them interchangeable	field_merge.py — one merge function per field, SOURCE_PRIORITY picks the winner	8
Factory Method · GoF	Lets a creator decide which concrete class to instantiate	get_store() picks the concrete store; callers never build one	3
Repository · Fowler	Callers need records, not storage details	BaseDataStore hides files today, a database later	2
Dependency Injection · Fowler	High-level code needs a part without building it	FastAPI Depends on the server; the provided client in the UI	3, 4
Composite · Decorator · State · Observer · GoF	Tree structures, added responsibilities, behavior that follows state, change notification	The component tree · the DataState wrapper · the decide flow · controlled forms	4 to 7
Provider · Hook · Container · React idioms	Screens share a client, logic, and data duties	App entry · src/lib/ · Screens	4, 5



The stack from View 5, with each pattern pinned to the layer where it lives.

Adapter · GoF
A normaliser per source converts source fields to the one contract shape.

Template Method · GoF
The job skeleton is fixed; queriers and normalisers provide the steps.

Strategy · GoF
One merge function per field; SOURCE_PRIORITY picks the winner.

Repository · Fowler
BaseDataStore hides where records persist from every caller.

Dependency Injection · Fowler
Concrete parts are built at composition points, passed in as parents.

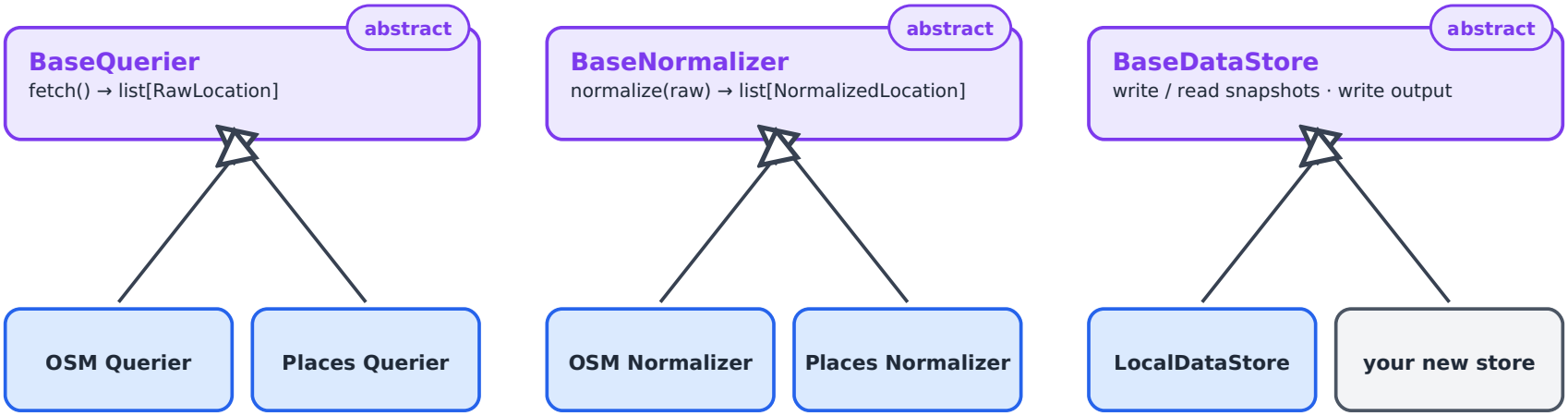
Factory Method · GoF
get_store() decides which concrete store to instantiate.

Composite · Decorator · State · GoF
The UI's tree, its wrappers, and flows that follow state.

Figure 1 — The pattern map. The layer stack of View 5 with each pattern pinned to the layer where it lives.

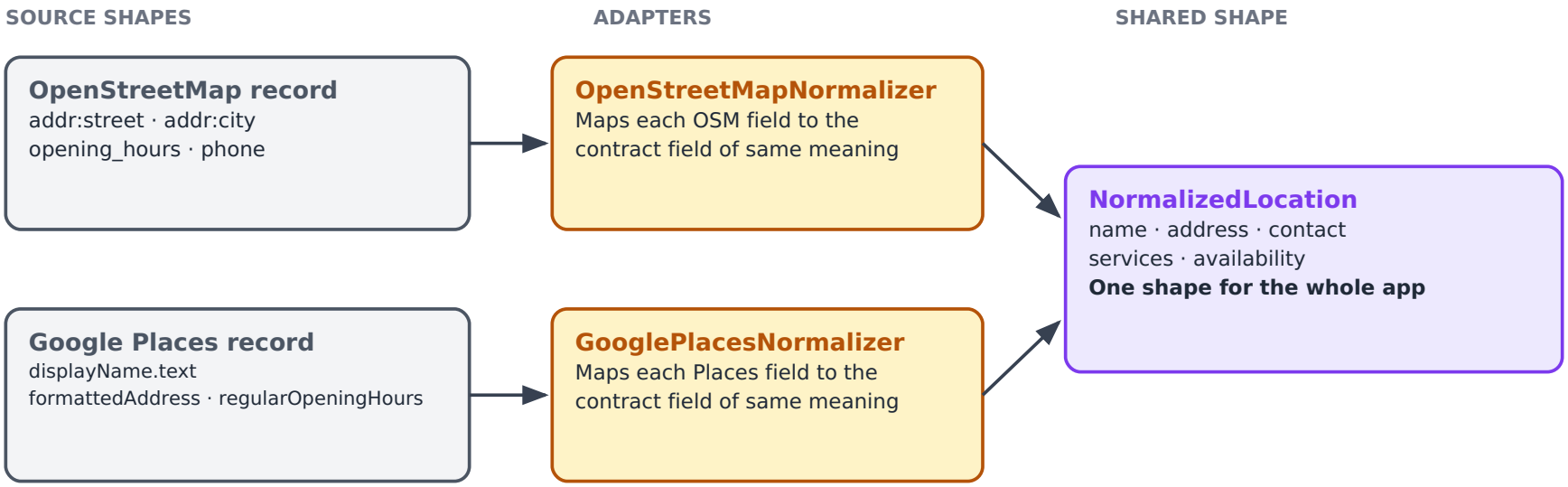
Module rule: one source, one folder. A source lives in etl/src/etl/sources/<name>/ and holds one querier.py and one normalizer.py. Nothing else goes in that folder.

Parents and adapters — extend by adding a child



Add a part: write a child. Change a part: swap the child. Callers never change.

Figure 2 — The three parents and their children. Hollow arrows point from a child to its parent: the child fills the parent shape.



A field a source does not have stays empty. The adapter never invents a value.

Figure 3 — The adapter pattern. Two source shapes enter; one contract shape leaves. Each normaliser is an adapter.

Stub 1 — Add a data source (ETL)

```
# etl/src/etl/sources/example_source/querier.py
from etl.base.querier import BaseQuerier
from etl.dtos import RawLocation

class ExampleSourceQuerier(BaseQuerier):
    """Reads locations from Example Source."""
    def fetch(self) -> list[RawLocation]:
        # 1. Call the source API. Handle paging inside this method.
        # 2. Return raw records. Do not map fields here.
        ...

# etl/src/etl/sources/example_source/normalizer.py
from contracts.domain import NormalizedLocation
from etl.base.normalizer import BaseNormalizer
from etl.dtos import RawLocation

class ExampleSourceNormalizer(BaseNormalizer):
    """Maps Example Source fields to the shared schema."""
    def normalize(self, raw_locations: list[RawLocation]) -> list[NormalizedLocation]:
        # Map each source field to the contract field of the same meaning.
        # A field the source does not have stays empty. Do not invent values.
        ...

# in a job script — the composition point, modeled on jobs/scrape_to_local.py
store = LocalDataStore(data_dir)            # callers see the parent: BaseDataStore
raw = ExampleSourceQuerier().fetch()
store.write_source_snapshot(DataSource.EXAMPLE_SOURCE,
                            ExampleSourceNormalizer().normalize(raw))
```

Stub 2 — Add a storage backend (Repository)

```
# packages/data-store — a new backend is a new child of the same parent
class DatabaseDataStore(BaseDataStore):
    def __init__(self, connection_url: str) -> None:
        ...  # store the setting; open nothing yet
    def write_source_snapshot(self, source, normalized_locations) -> None: ...
    def read_source_snapshot(self, source): ...
    def write_output_locations(self, output_locations) -> None: ...
# No caller changes. Jobs and the API depend on BaseDataStore.
```

The run skeleton, merge rules, and injection

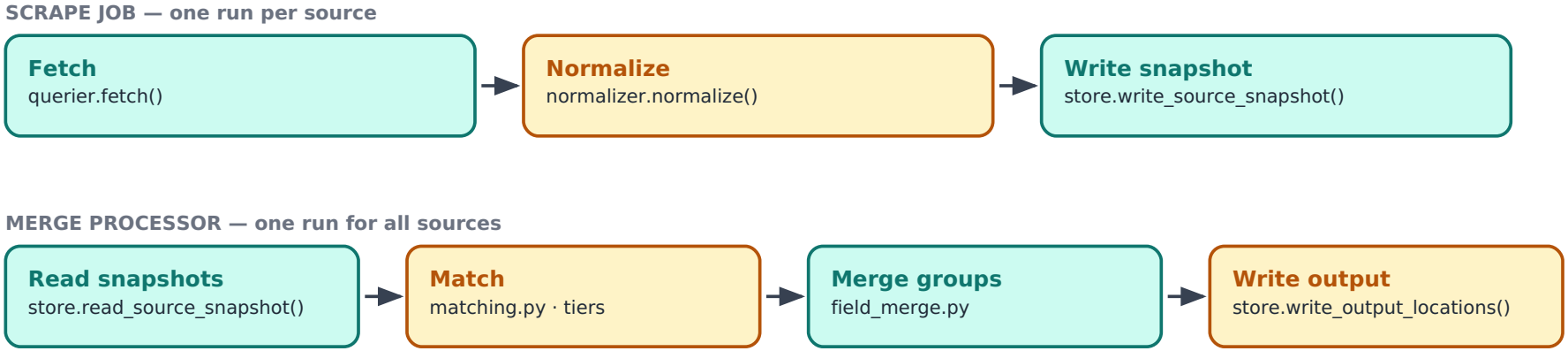


Figure 4 — Template Method. Both jobs run a fixed skeleton in a fixed order; the children provide the steps, so a new source changes no step.

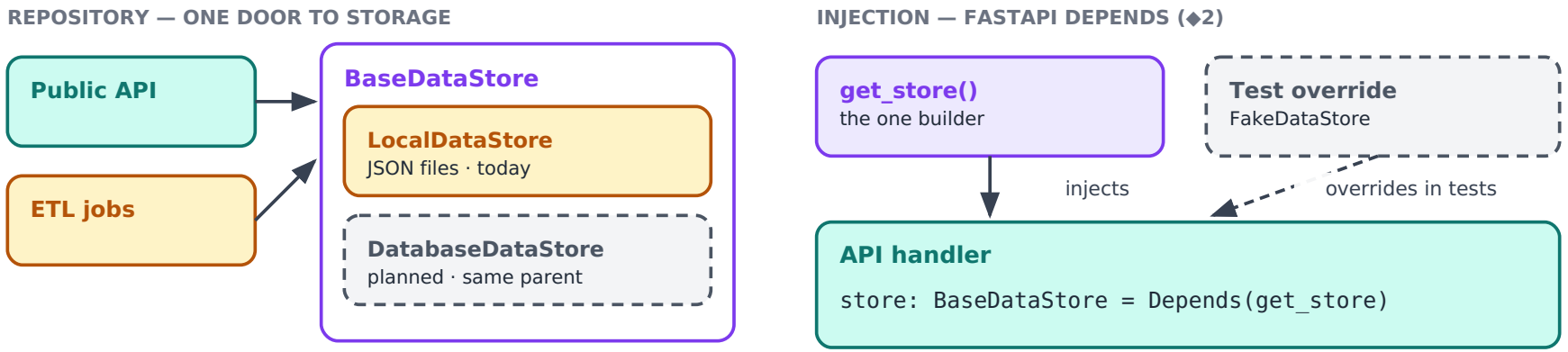


Figure 5 — The strategy pattern in field_merge.py. One function per field; SOURCE_PRIORITY settles every conflict.

Stub 8 — Add a field-merge rule

```
# etl/src/etl/field_merge.py — one new function, one new line in merge_group
def _merge_access_notes(notes: list[str | None]) -> str | None:
    # One rule, stated once: take the value from the highest-priority
    # source that has one. SOURCE_PRIORITY is the order of trust.
    # Never pick by record order, and never blend two values.
    ...

# Thresholds are named constants at the top of the file, as in matching.py:
# MAX_MATCH_DISTANCE_M = 150 · NAME_SIMILARITY_THRESHOLD = 90. No bare numbers.
```



Callers name the parent type. The concrete store is chosen in one place, so a swap touches one function — and tests swap it the same way.

Figure 6 — Repository on the left, injection on the right. Handlers and jobs name the parent; one builder picks the concrete store; tests override the same builder.

Stub 3 — Add an API endpoint (lands with #49)

```
# server — pseudo-code for the #49 endpoints
from fastapi import Depends
from data_store.base import BaseDataStore

def get_store() -> BaseDataStore:
    # The one place that picks the concrete store.
    return LocalDataStore(Path(settings.data_dir))

@router.get("/locations", response_model=LocationList)
async def list_locations(store: BaseDataStore = Depends(get_store)):
    # The handler calls the business layer and returns its result.
    # Rules do not live in the handler.
    ...

# in a test — swap the store, keep the handler
app.dependency_overrides[get_store] = lambda: FakeDataStore()
```

The UI does not inherit — it composes

React gets its reuse from composition and hooks, not class parents.

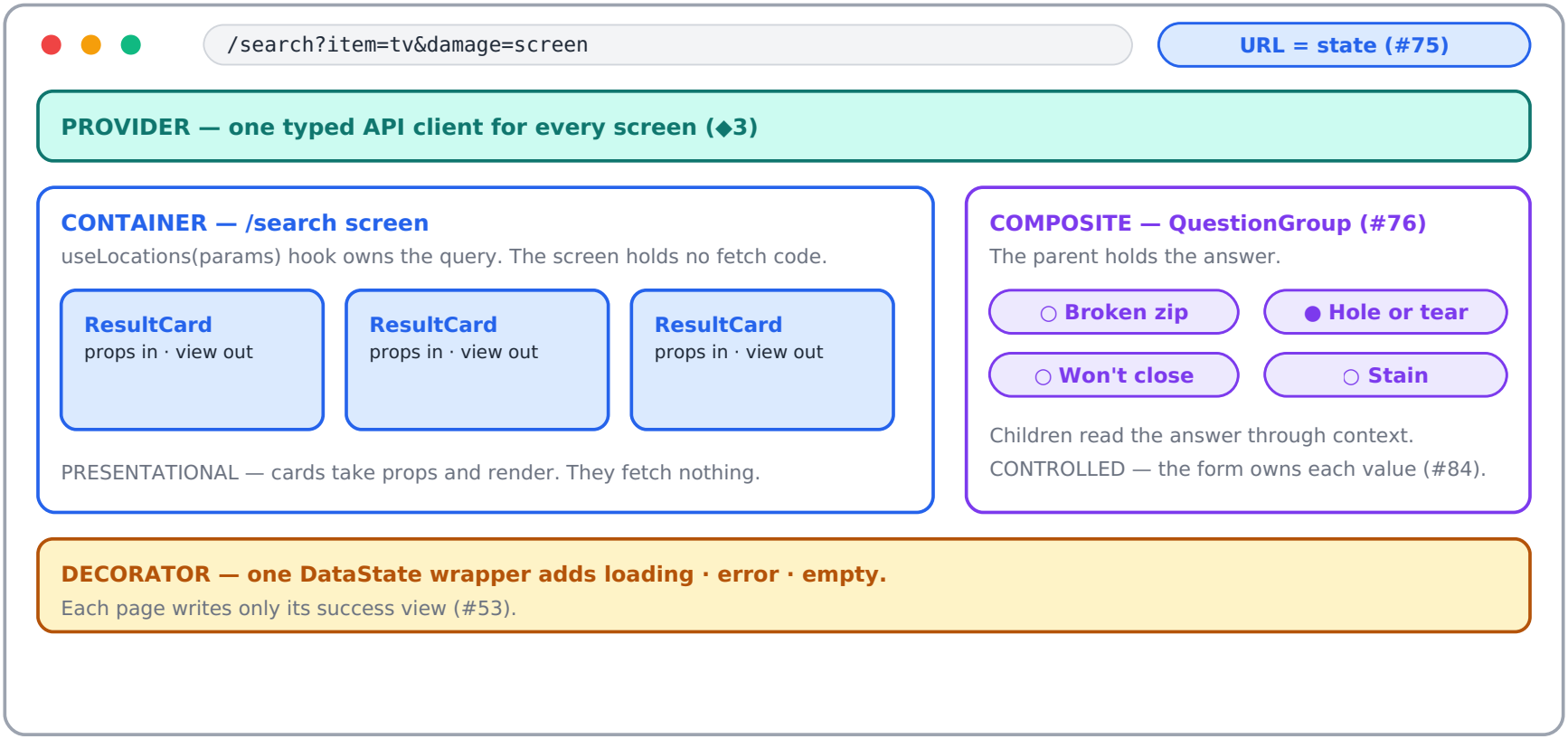


Figure 7 — Anatomy of a screen. The provider supplies the client; the container owns the query; presentational cards render props; the URL carries shareable state.

Pattern	Rule
Provider · React idiom	One typed API client, built at the entry, reaches every screen through context (◆3)
Container / presentational · React idiom	A component that fetches is a container. A component that only renders takes props. Never both in one file
Custom hook · React idiom	Reusable logic sits in a hook, so screens stay thin and logic is testable alone
Composite · GoF	A part-whole tree where parents and children share one interface: a question group holds its options the way a page holds its parts (#76)

Decorator · GoF	A wrapper adds responsibilities around a component: DataState renders loading, error, and empty around the success view (#53)
Controlled component · Observer shape, GoF	The parent form owns the value; children update when the state changes and notifies them (#84)
URL as state	Shareable state lives in the URL, not in a component (#75)
State · GoF	The flow alters its behavior when its state changes: the next step is a function of the answers so far (#76)

State lives at the lowest parent that needs it — the URL for shareable state, the form for form state, the hook for query state. Do not lift state to the app root by default.

Stubs 4 to 7 — Service, hook, compound group, controlled input

```
// Stub 4 — a service (client/src/lib/locations.ts). The only place that calls the API.
export async function getLocations(params: SearchParams): Promise<Location[]> {
  // Use the typed client provided at the app entry. Return contract types (#46).
  // No page writes its own fetch.
}

// Stub 5 — a hook over the service (client/src/lib/use-locations.ts)
export function useLocations(params: SearchParams) {
  // Wrap the service call in the app's query tool.
  // Return { data, isLoading, error } in one shape for every screen.
}

// Stub 6 — a compound question group (decide flow, #76). The parent holds
// the answer; children read it through context, never as props from the page.
<QuestionGroup id="damage">
  <QuestionOption value="zip">Broken zip</QuestionOption>
  <QuestionOption value="hole">Hole or tear</QuestionOption>
</QuestionGroup>

// Stub 7 — a controlled input (decide forms, #84). The form owns the value.
<ItemInput value={item.name} onChange={setItemName} />
// If a parent needs the value to decide the next step, the component is
// controlled. Local-only text can stay uncontrolled.
```

THE RULES UNDER THE PATTERNS

SOLID, in its authors' words

Five class-design principles from Robert C. Martin, “Design Principles and Design Patterns” (2000); the acronym is Michael Feathers’. OCP is Bertrand Meyer’s; LSP is Barbara Liskov’s (1987).

S

Single Responsibility

“A class should have one, and only one, reason to change.”

Here: a class changes when its one source changes, or its one contract changes — never both.

O

Open / Closed

“Software entities should be open for extension, but closed for modification.”

Here: extend by adding a child. The pipeline and the callers do not change.

L

Liskov Substitution

“Objects should be replaceable with instances of their subtypes without altering the correctness of the program.”

Here: every store child stands in for LocalDataStore, in tests and in production.

I

Interface Segregation

“Clients should not be forced to depend on interfaces they do not use.”

Here: parents hold one to three methods. Public (#46) and internal (#47) contracts stay separate.

D

Dependency Inversion

“High-level modules should not depend on low-level modules; both should depend on abstractions.”

Here: seams ◆1 to ◆4. High-level code names parents; builders supply the concrete parts.

One honest limit: the jobs still wire each source by name (scrape_to_local.py builds each querier in turn). Open/Closed holds for the pipeline classes; the job file grows a few lines per source. A source registry closes that gap when a third source arrives — not before.

Change a contract field

A contract change touches both sides, so it has an owner and an order: propose the change to the contract owner (#46 API: Yuan · #75 search URL: Hubert) → change packages/contracts and the API schema in one pull request → update the ETL writer and the API reader → regenerate the client types from the API schema; never edit them by hand.

What not to do

- Do not build a concrete store, querier, or client inside a handler, a page, or a rule. Build it in a job or an entry file, and pass the parent type in.
- Do not add a field to a contract on one side only.
- Do not put a business rule in a router or a page. Rules live in the business layer (View 5).
- Do not add a dependency injection container. Constructor injection and FastAPI Depends cover every seam in this app.

Grounded in codeforboston/boston-circular-economy · Oct 2026 · Companion to architecture Views 1 to 5 · Pattern sources: Gamma, Helm, Johnson, Vlissides, “Design Patterns” (1994); Fowler, “Patterns of Enterprise Application Architecture” (2002) and “Inversion of Control Containers and the Dependency Injection Pattern” (2004) · SOLID sources: Martin (2000); DigitalOcean, “S.O.L.I.D — The First Five Principles of Object-Oriented Design.”